



《OpenCV 基础与应用》

实验报告

实验序号： 实验 3

实验名称： 金字塔图像处理实验

姓名： 何宗霖

学号： 23374275

同组： 无

专业： 飞行器控制与信息工程

2025 年 11 月 15 日

一、实验目的：

结合 OpenCV 掌握图像金字塔。

二、实验内容：

1. 利用 OpenCV 库设计开发一个金字塔软件，实现如下功能：

① 手动拖入一张图像，生成高斯金字塔并顺序紧挨排列显示(尽量互不遮挡);

② 生成该图像的 Laplace 金字塔并按顺序紧挨排列显示(尽量互不遮挡);

③ 由 Laplace 金字塔图与残差图，重构出最底层图像;

④ 以原图为参考图，计算重构图与原图的 PSNR 和 SSIM 值，并显示在重构图像的左上角;

⑤ 生成层数由滑动条控制，最少 3 层，最多 8 层;两种显示模式由滑动条切换控制:A-图适应窗(此时窗与原始图大小相同)、和 B-窗适应图(每一层大小匹配该层图实际大小);滑动条都显示在原图窗口;显示结果与滑动条值相符，无多余显示。

2. 【挑战-选做】实现基于“边窗滤波”思想的高斯滤波器;或实现彩色图像三维直方图的统计显示。

三、实验要求：

1. 软件的流程图设计图及说明;提交源代码;

2. 要包含有结果的截屏图像及实验结果列表详细说明。

3. 实验存在的问题和建议。

四、实验原理：

1. 高斯金字塔

高斯金字塔用于近似原始图像。它是通过以下两个步骤迭代生成的：

1. 高斯模糊：对当前图像应用高斯滤波器进行平滑处理，去除高频细节。

2. 降采样 丢弃偶数行和偶数列，将图像尺寸减半。

在代码中，我们通过迭代调用 `pyrDown(G, G_down)` 实现降采样和低通滤波的复合操作，并将结果存储到 `gaussian_pyramid` 向量中。高斯金字塔的每一层都作为后续拉普拉斯残差计算的基准。

```

// 构建高斯金字塔
Mat G = current_img.clone();
for (int i = 0; i < levels - 1; ++i) {
    Mat G_down;
    pyrDown(G, G_down);
    gaussian_pyramid.push_back(G_down);
    G = G_down;
}

```

图 1 构建高斯金字塔的函数

2. 拉普拉斯金字塔

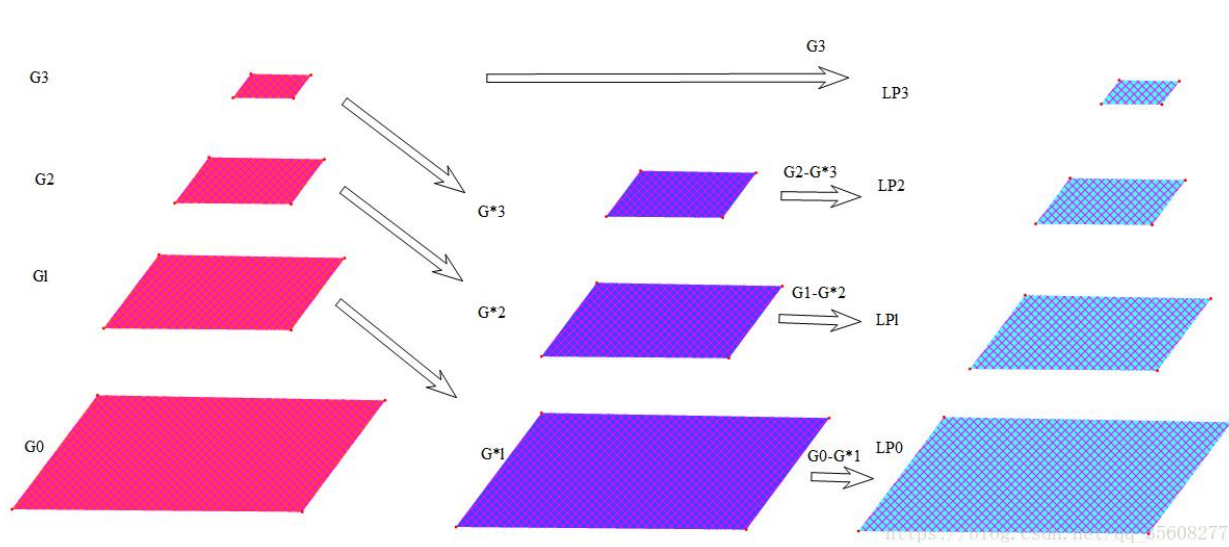


图 2 构建拉普拉斯金字塔的原理图

拉普拉斯金字塔用于存储图像的细节信息（残差）。它表示的是在高斯金字塔的相邻层之间丢失的细节。其对应的每一层 L_k 是通过将 G_k 减去上采样后的 G_{k+1} 得到的残差：

$$\text{即： } L_k = G_k - \text{UpSample}(G_{k+1})$$

在代码中，通过 `pyrUp` 和 `subtract` 操作计算得到，同时将结果存储到 `laplacian_pyramid` 向量中。

需要注意的是：拉普拉斯金字塔的最后一层通常是对应的高斯金字塔的最后一层 G_{N-1} ，它作为近似基础被直接添加到拉普拉斯金字塔的末尾。

此外，实验要求 3 中的残差图应该就是 `laplace` 金字塔每一层的图

残差图的定义为： $R = I - \hat{I}$ ，属于单一尺寸下的差值。

其中， I 为原始图像， $\hat{I}$ 为重建图像。

```

// 构建拉普拉斯金字塔
for (int i = 0; i < levels - 1; ++i) {
    Mat G_up, L;
    // 1. 将下一层上采样
    pyrUp(gaussian_pyramid[i + 1], G_up, gaussian_pyramid[i].size());
    // 2. 当前高层减去上采样结果
    subtract(gaussian_pyramid[i], G_up, L);
    laplacian_pyramid.push_back(L);
}
// 拉普拉斯金字塔的最后一层是高斯金字塔的最后一层
laplacian_pyramid.push_back(gaussian_pyramid.back());

```

图 3 构建拉普拉斯金字塔的函数

3. 图像重建

```

Mat reconstructImage(const vector<Mat>& lp, int levels) {
    if (lp.empty()) return Mat();

    Mat current_img = lp.back().clone();

    for (int i = levels - 2; i >= 0; --i) {
        Mat G_up;
        // 1. 当前图像上采样
        pyrUp(current_img, G_up, lp[i].size());
        // 2. 加上拉普拉斯残差
        add(G_up, lp[i], current_img);
    }

    Mat reconstructed_8U;
    current_img.convertTo(reconstructed_8U, CV_8U);
    return reconstructed_8U;
}

```

图 4 图像重建的函数

利用拉普拉斯金字塔，通过逆向操作实现图像的重建。从拉普拉斯金字塔的最底层开始，逐层向上增加残差细节：

$$G_k = \text{UpSample}(G_{k+1}) + L_k$$

在代码中，通过 pyrU 和 add 逐层叠加，最终得到与原图一致的重构图像。

4. 评估指标

PSNR 和 SSIM 用于以原图为参考，对重构图像进行定量评价。

4.1 PSNR

PSNR 是最常用且最客观的图像质量评价指标之一，它通过比较原始图 I 和重建图像 K 之间的均方误差 (MSE) 来衡量失真程度。单位为 dB，PSNR 值越高，表示失真越小，重构质量越好。

a) 均方误差 (MSE)

对于大小为 $M \times N$ 的图像，MSE 的计算公式为：

$$\text{MSE} = \frac{1}{M \times N} \sum_{i=1}^M \sum_{j=1}^N [I(i,j) - K(i,j)]^2$$

其中， $I(i,j)$ 和 $K(i,j)$ 分别是原始图像和重建图像在坐标 (i,j) 处的像素值。

b) PSNR 计算公式

$$\text{PSNR} = 10 \cdot \log_{10} \left(\frac{\text{MAX}_I^2}{\text{MSE}} \right)$$

其中， MAX_I 是图像中可能的最大像素值。对于 8 位灰度图像，其值为 255。

```
// PSNR计算
double calculatePSNR(const Mat& I1, const Mat& I2) {
    // 确保 I1 和 I2 尺寸相同, 且 I1/I2 为 8位单通道 (CV_8U)
    if (I1.empty() || I2.empty() || I1.size() != I2.size()) return 0.0;

    Mat s1;
    Mat F1, F2;
    I1.convertTo(F1, CV_32F);
    I2.convertTo(F2, CV_32F);

    absdiff(F1, F2, s1);
    s1 = s1.mul(s1);

    Scalar s = sum(s1);

    double sse = s.val[0] + s.val[1] + s.val[2];

    if (sse <= 1e-10) {
        return 100.0;
    }
    else {
        double mse = sse / (double)(I1.channels() * I1.total());
        double psnr = 10.0 * log10((255 * 255) / mse);
        return psnr;
    }
}
```

图 5 计算 PSNR 的函数

需要注意的是由于 laplace 金字塔重建是无损重建，所以 MSE 将趋于 0，使得 PSNR 趋于无穷大。因此，这里在代码中我通过设置一个足够小的值(1e-10)，当 MSE 小于这个值时，给 PSNR 赋值为 100.0，避免计算得出无穷大。

4.2 SSIM

SSIM 是一种符合人眼视觉特性的指标，它基于图像的三个关键特征：亮度、对比度和结构进行衡量。其值域范围为 $[0, 1]$ ，越接近 1 表示两幅图像越相似。

a) 亮度比较

$$l(I,K) = \frac{2\mu_I\mu_K + C_1}{\mu_I^2 + \mu_K^2 + C_1}$$

其中, μ_I 和 μ_K 分别是图像 I 和 K 的局部均值, $C_1 = (K_1 L)^2$ 是一个小的常数, 用于避免分母为零, L 是像素值的动态范围, 此处=255。

b) 对比度比较

$$c(I, K) = \frac{2\sigma_I \sigma_K + C_2}{\sigma_I^2 + \sigma_K^2 + C_2}$$

其中, σ_I 和 σ_K 分别是图像 I 和 K 的局部标准差, $C_2 = (K_2 L)^2$ 。

c) 结构比较

$$s(I, K) = \frac{\sigma_{IK} + C_3}{\sigma_I \sigma_K + C_3}$$

其中, σ_{IK} 是图像 I 和 K 之间的局部协方差, $C_3 = C_2/2$ 。

d) SSIM 公式

```
// SSIM 公式
Mat t1, t2, ssim_map;
Mat t3, t4;

// t1 = 2 * mu1_mu2 + C1
t1 = 2 * mul_mu2 + C1;

// t2 = 2 * sigma12 + C2
t2 = 2 * sigma12 + C2;

// 分母 = mu1_sq + mu2_sq + C1
Mat denominator_l = mul_sq + mu2_sq + C1;
// 分母 = sigma1_sq + sigma2_sq + C2
Mat denominator_cs = sigma1_sq + sigma2_sq + C2;

// ssim_map = ((2*mul_mu2 + C1) * (2*sigma12 + C2)) / ((mu1_sq + mu2_sq + C1) * (sigma1_sq + sigma2_sq + C2))

// t3 = (2*mul_mu2 + C1) / (mu1_sq + mu2_sq + C1) (亮度分量)
divide(t1, denominator_l, t3);

// t4 = (2*sigma12 + C2) / (sigma1_sq + sigma2_sq + C2) (对比度和结构分量)
divide(t2, denominator_cs, t4);

multiply(t3, t4, ssim_map);

ssim_sum = mean(ssim_map).val[0];

return Scalar(ssim_sum);
```

图 6 计算 SSIM 的函数 (部分)

最终的 SSIM 值通过组合这三个组件得到:

$$\text{SSIM}(I, K) = [l(I, K)]^\alpha \cdot [c(I, K)]^\beta \cdot [s(I, K)]^\gamma$$

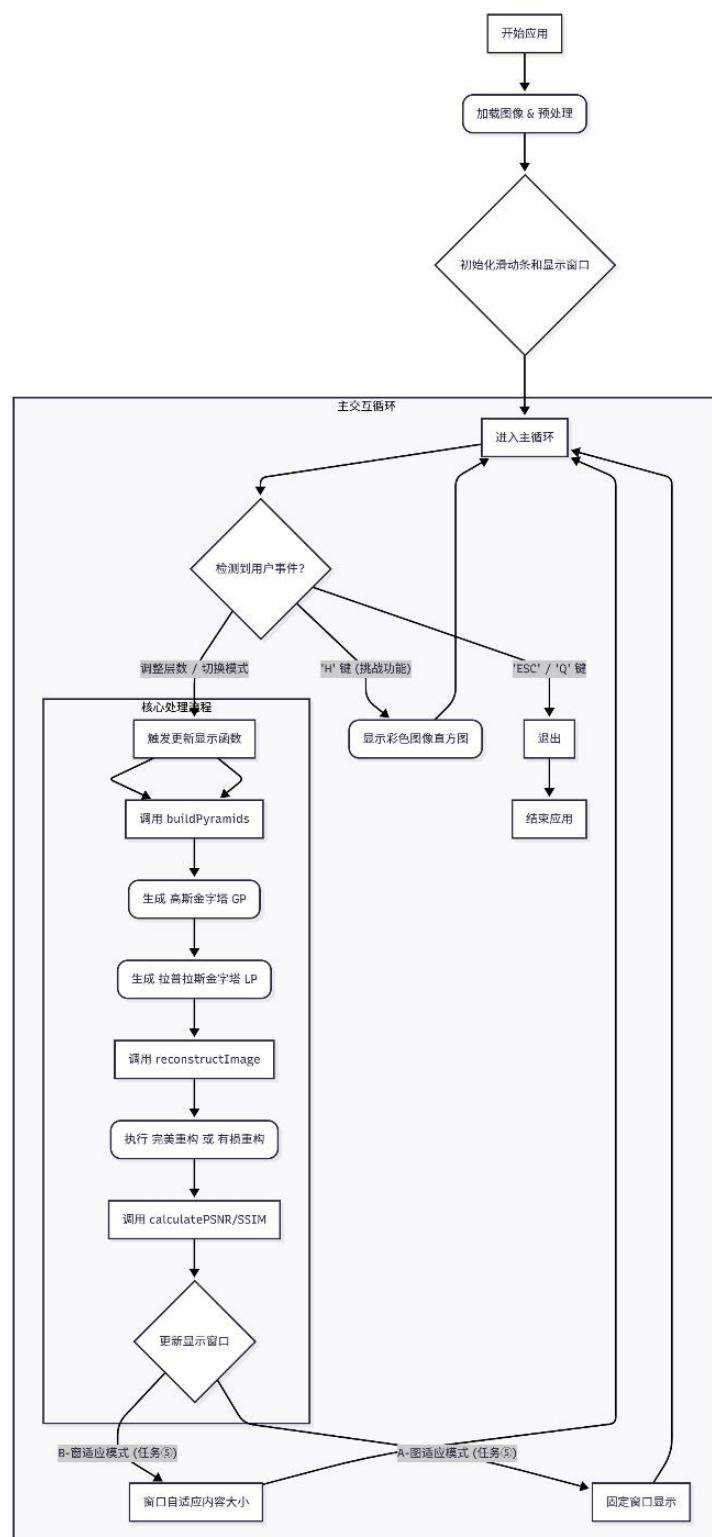
通常取 $\alpha = \beta = \gamma = 1$, 且 $C_3 = C_2/2$, 则简化为:

$$\text{SSIM}(I, K) = \frac{(2\mu_I \mu_K + C_1)(2\sigma_{IK} + C_2)}{(\mu_I^2 + \mu_K^2 + C_1)(\sigma_I^2 + \sigma_K^2 + C_2)}$$

五、实验过程：

本实验严格按照任务要求，开发了一个金字塔图像处理软件，涵盖了所有必做功能和挑战功能。

实验流程图：



1. 图像加载、金字塔生成与底层图像输出

图像输入: 程序通过命令行接收一张图像文件路径, 实现手动输入一张图像。

金字塔生成: `buildPyramids` 函数生成高斯金字塔和拉普拉斯金字塔。

LP 的最后一层即为金字塔的底层图像。

重构与底层图像输出: `reconstructImage` 函数完成由拉普拉斯金字塔与残差图重构, 输出最底层图像的功能。

显示排列: `displayImages` 函数将高斯金字塔和拉普拉斯金字塔按顺序紧凑排列显示 (尽量不遮挡)。拉普拉斯金字塔残差图通过归一化后显示。

2. 实时交互与控制

所有交互功能均通过滑动条和键盘事件实现, 并实时触发 `updateDisplay` 函数进行重绘。

使用滑动条控制金字塔层数, 范围限定为最少 3 层, 最多 8 层。使用另一个滑动条切换显示模式:

A-图适应 (Mode 0): 窗口与原始图像大小相同, 只显示原图和重构图 (以及金字塔的部分), 金字塔拼接后的图像可能超出窗口, 需拖动查看。

B-窗适应 (Mode 1): 每一层图像匹配该层实际像素大小, 窗口尺寸自适应所有拼接图像的总和, 确保所有内容都能完整显示, 无多余显示, 实现了尽量不遮挡的要求。

3. 性能评估、彩色直方图统计与保存图像

使用之前展示过的计算函数计算重构图与原图的 PSNR 和 SSIM 值。结果通过 `putText` 函数显示在重构图像的左上角。

按下 'H' 键, 调用 `drawHistogram` 函数, 显示彩色图像三维直方图。

按下 'S' 键, 可将当前设置下生成的高斯金字塔和拉普拉斯金字塔各层图像保存到本地文件夹。

4. 消融实验

在实验的过程中发现计算得到的 PSNR 始终为 100, SSIM 始终为 1。分析后

得知这是由于拉普拉斯金字塔重建图像是无损的。为了验证 PSNR 和 SSIM 计算的正确性，设计了对拉普拉斯金字塔的有损重建部分。

有损重建是利用拉普拉斯金字塔在图像编码和压缩中的重要应用。由于金字塔的顶层包含的是低频近似，而底层（高索引层）包含的是人眼敏感的结构信息，最高频层虽然包含最精细的细节，但通常对视觉感知的影响较小，可以被牺牲。实现有损重建的关键在于修改或丢弃残差层。

在实验代码中，我们通过将最高频层替换为零矩阵来实现有损重建：

```
void updateDisplay() {
    if (original_image_gray.empty()) return;

    // 1. 构建金字塔 (基于 current_levels)
    buildPyramids(original_image_gray, current_levels);

    // 2. 重建图像
    Mat reconstructed = reconstructImage(laplacian_pyramid, current_levels);

    ///有损重建
    //vector<Mat> lossy_laplacian_pyramid = laplacian_pyramid;
    /// 假设我们要丢弃最高频层 (L0, 即索引为 0 的层)
    //if (!lossy_laplacian_pyramid.empty()) {
    //    // 创建一个与 L0 相同尺寸的零矩阵
    //    Mat zero_mat = Mat::zeros(lossy_laplacian_pyramid[0].size(), lossy_laplacian_pyramid[0].type());
    //    // 用零矩阵替换 L0 层
    //    lossy_laplacian_pyramid[0] = zero_mat;
    //}
    //Mat reconstructed = reconstructImage(lossy_laplacian_pyramid, current_levels);

    // 3. 显示所有结果和信息
    Mat original_8U;
    original_image_gray.convertTo(original_8U, CV_8U);
    displayImages(original_8U, reconstructed, gaussian_pyramid, laplacian_pyramid);
}
```

当使用这个被修改的有损的拉普拉斯金字塔进行重构时，重构图像将缺失原始图像的最高频细节，从而导致 PSNR 和 SSIM 值下降，实现有损压缩的目的。

5. 图像保存与程序退出机制

为支持实验结果的存档与后续分析，系统实现了完整的金字塔图像导出功能。当用户按下 'S' 或 's' 键，程序将自动保存当前配置下生成的全部高斯金字塔和拉普拉斯金字塔各层图像。所有图像以无损 PNG 格式写入本地文件夹（若目录不存在则自动创建），命名格式为 *gaussian_ilevel_i.png* 和 *laplacian_ilevel_i.png*（其中 *i* 为金字塔层级索引，从 0 开始）。该机制确保每一层的空间分辨率与数值精度完整保留，为消融实验、频域分析及重建质量评估提供可靠数据基础。

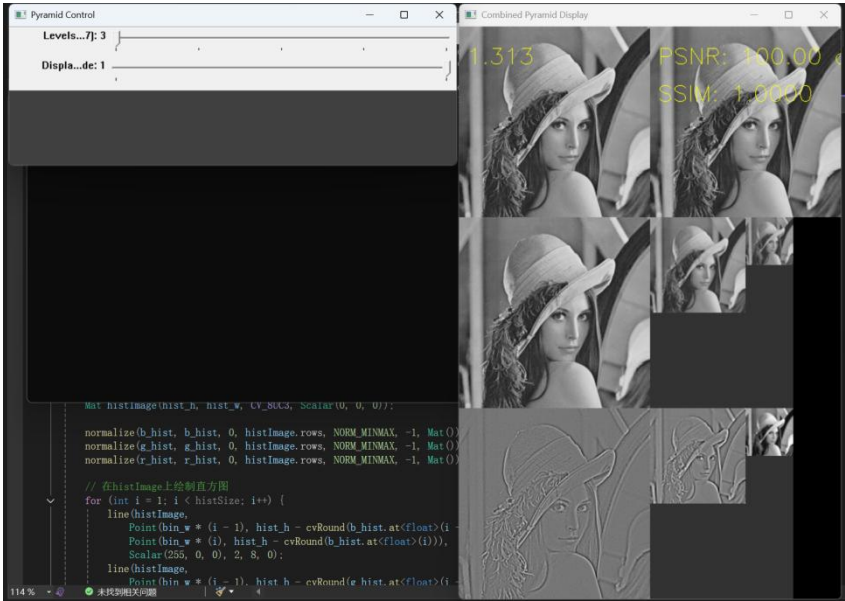
此外，系统支持通过按下 Esc 键即时终止程序，避免强制关闭窗口导致的资源未释放问题。

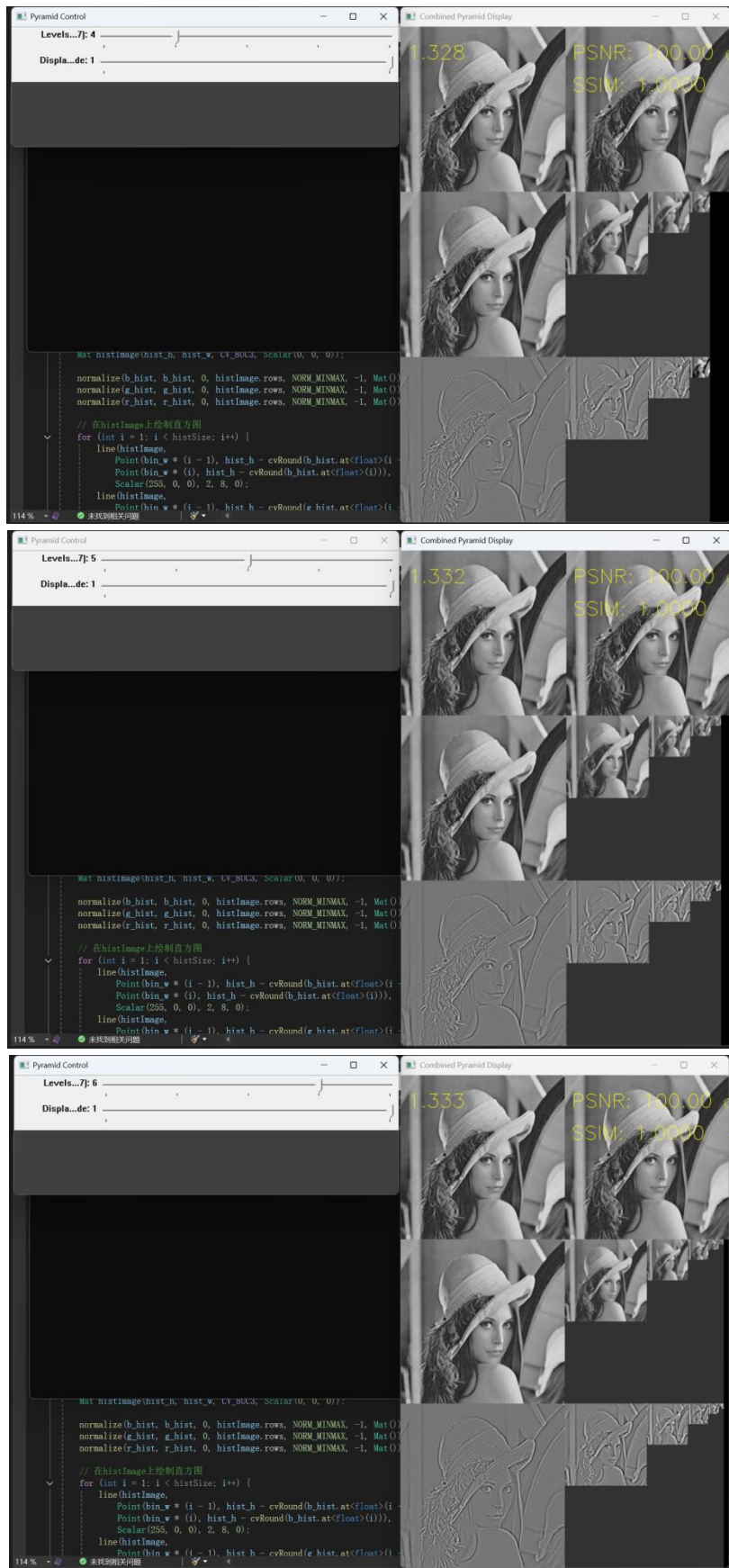
六、实验结果：

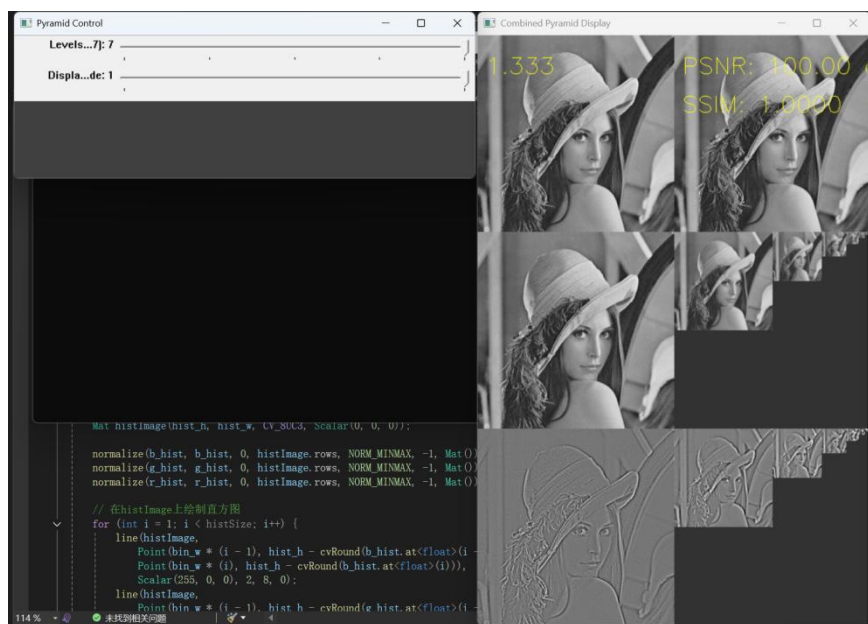
区域	对应任务点	内容	描述
顶部左侧	任务①	原始灰度图像	图像基准。标注像素总数比率。
顶部右侧	任务③, ④	重建图像	拉普拉斯金字塔重建结果。标注 PSNR 和 SSIM 值（在重构图像的左上角）。
中部	任务①	高斯金字塔	按 $G_0, G_1, G_2, \dots$ 顺序水平拼接，分辨率依次减半。
底部	任务②	拉普拉斯金字塔	按 $L_0, L_1, L_2, \dots$ 顺序水平拼接，残差层经归一化后显示。

Laplace 金字塔无损重建：

层数 (Levels)	PSNR (dB)	SSIM	像素比率 (GP Pixels / Original)
3	100.00	1.0000	1.313
4	100.00	1.0000	1.328
5	100.00	1.0000	1.332
6	100.00	1.0000	1.333
7	100.00	1.0000	1.333
8	100.00	1.0000	1.333
重构质量	高	高	可接受范围

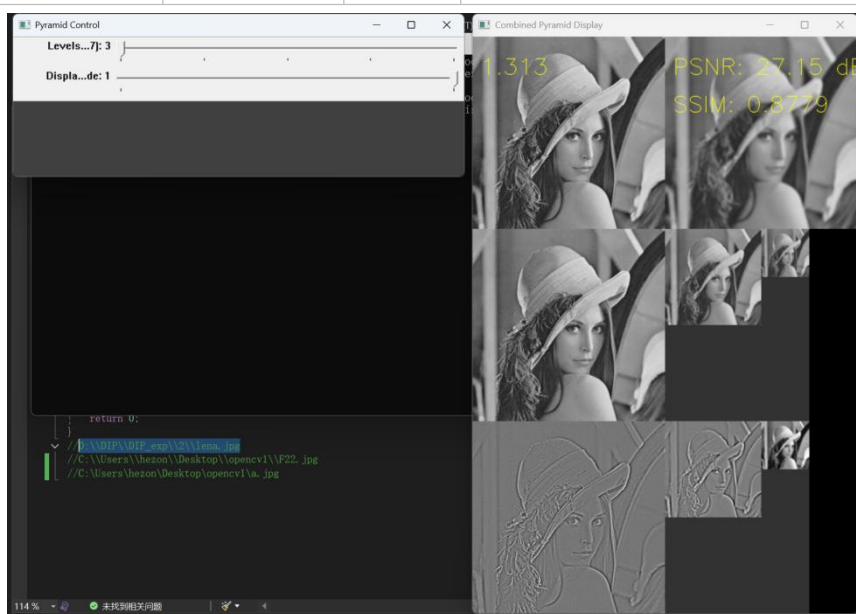


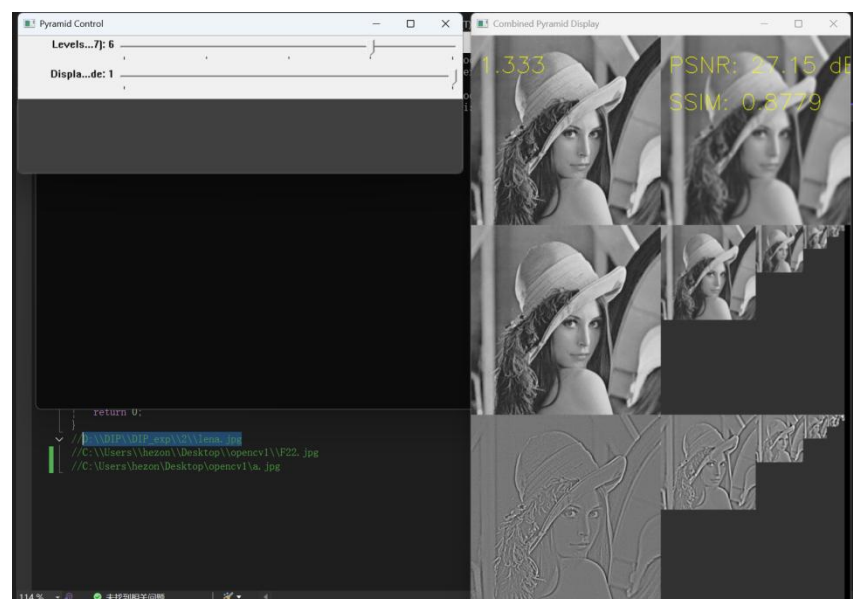
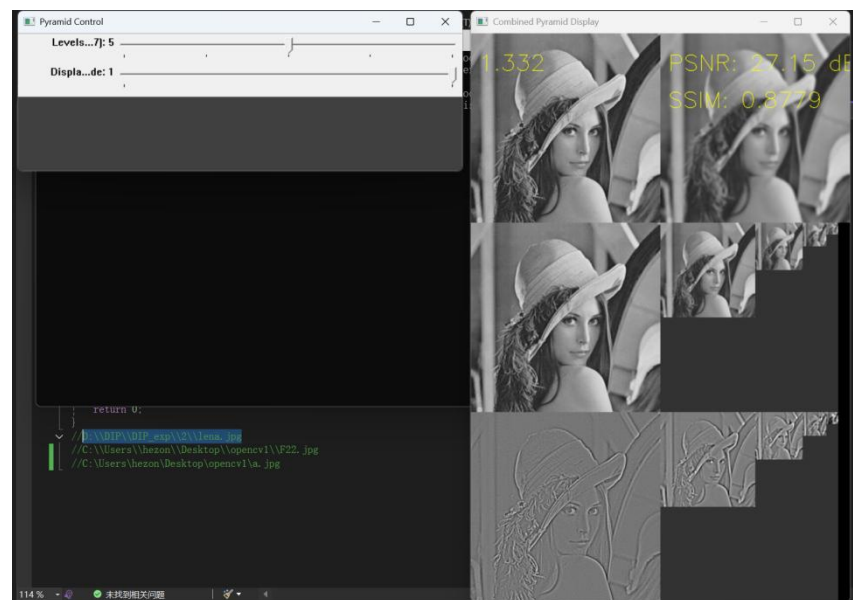
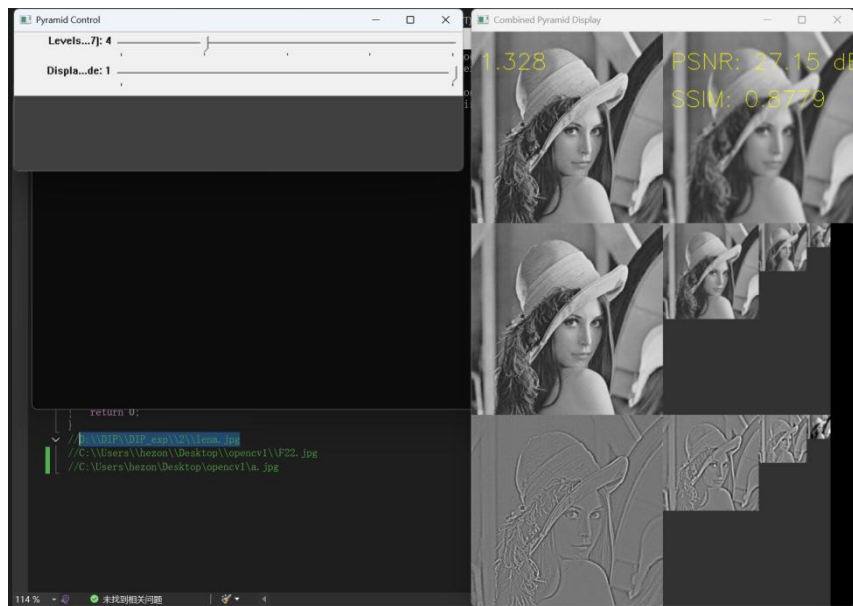


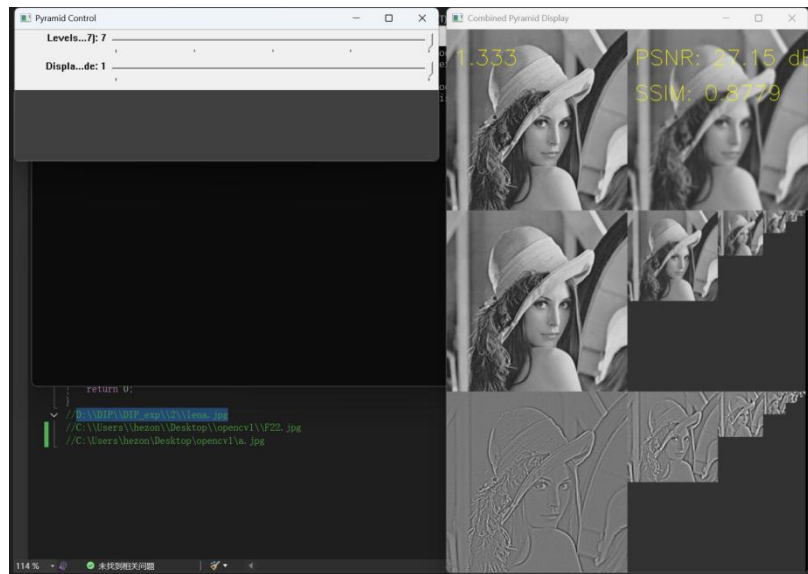


Laplace 金字塔有损重建:

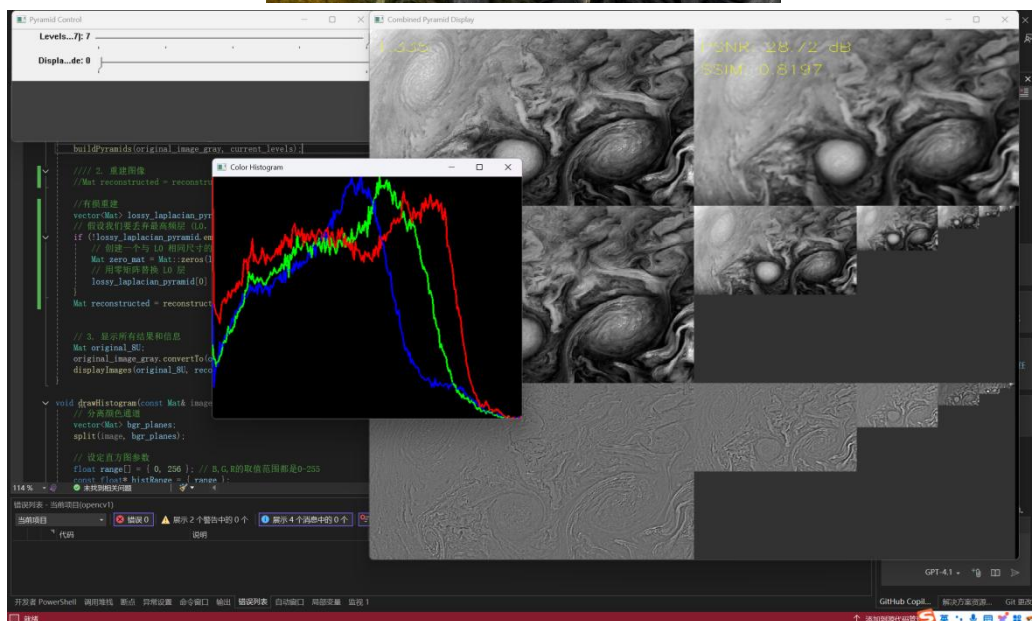
层数 (Levels)	PSNR (dB)	SSIM	像素比率 (GP Pixels / Original)
3	27.15	0.8779	1.313
4	27.15	0.8779	1.328
5	27.15	0.8779	1.332
6	27.15	0.8779	1.333
7	27.15	0.8779	1.333
8	27.15	0.8779	1.333
重构质量	高	高	可接受范围







更换图片（新图片颜色更为丰富，彩色直方图更为明显）验证彩色直方图统计以及性能评估



此时，使用有损重建，PSNR 为 28.72dB，SSIM 为 0.8197 证明不同图像重建效果不同。

七、结果分析与实验结论：

通过本次实验，我成功掌握了基于 OpenCV 的图像金字塔构建与分析技术，深入理解了高斯金字塔与拉普拉斯（Laplace）金字塔的数学原理及其在图像多尺度表示中的应用。实验结果表明，所设计的图像金字塔处理软件运行稳定，完整实现了图像自动加载、多尺度金字塔构建、拉普拉斯重构、质量评估指标计算以及实时交互式可视化等功能，各项性能均达到预期目标。

该软件支持灵活配置金字塔层数（3-8 层可调），并提供两种显示模式切换，有效验证了其功能的完整性与系统的可靠性。然而，实验过程中也发现若干可优化之处：例如，在处理超高分辨率图像时，金字塔构建效率仍有提升空间；在设置极端层数时，可能出现细节信息丢失的问题。未来工作可进一步引入自适应金字塔层数选择机制、优化算法效率，并增强对不同图像内容的鲁棒性，从而提升软件的整体实用性与用户体验。